

TRACE 测试计划

属性	内容
文档版本	v2.4
状态	正式发布
适用版本	TRACE V1 — V2.4
文档负责人	加鹿鸣
审核人	肖弘基
最后更新	2026-06-24

修订历史

版本	日期	修订内容	修订人
v0.1	V1 阶段	初稿，覆盖工具层、API 基础和 MockLLM E2E	加鹿鸣
v1.0	V2.1 阶段	增加评测体系、experiment runner 测试策略	加鹿鸣
v2.0	V2.2 阶段	增加执行平台合同、source context、Contract Guard 专项	加鹿鸣
v2.3	V2.3 阶段	增加 Dataset authoring、mutation、probe confirmation 章节	加鹿鸣
v2.4	V2.4 阶段	增加前端 Playwright 自动化、Docker Compose 交付测试、分层流水线规划	加鹿鸣

术语与缩写

术语	含义
Clean Run	对原始项目快照执行测试生成与运行，用于建立 false positive 基线
Variant Replay	对 bug variant 执行冻结测试集（不调用 LLM），验证测试能否捕获已知缺陷
Capture Rate	bug variant 中被有效捕获的比例，分母仅含 confirmed + probe passed 的 variant
False Positive Rate	clean baseline 中出现失败的比例
Contract Guard	对生成测试进行结构化可信度验证的质量关卡

术语	含义
MockLLM	可离线运行的模拟 LLM，用于确定性流程回归
Probe Confirmation	对 auto mutation variant 进行 clean/buggy 预期差异验证，确认变异有效
metric_status	描述指标计算是否有效的状态字段（ok / invalid / error 等）
EvaluationEvent	记录评测过程中分类事件（provider_failure、invalid_test_set 等）的结构化日志
Replay Policy	控制 replay 执行并发、重试次数和退避时间的运行配置
P0/P1/P2/P3	测试用例和缺陷的优先级分级，P0 为最高优先级

1. 文档目的

本文档用于指导 TRACE 项目的测试设计、测试执行、缺陷跟踪和测试报告输出。测试计划覆盖后端服务、智能体运行时、工具层、执行器、评测体系、数据集管理、前端页面、Docker Compose 交付栈以及非功能质量要求。后续建立测试框架、编写测试用例、执行回归测试和生成项目测试报告，均以本文档为主要依据。

TRACE 的测试重点不同于普通管理系统。除常规接口、数据模型和前端页面外，还必须验证智能体生成测试的工程闭环、执行结果的结构化记录、反思修复的边界、评测指标的可信度，以及 replay 过程中不污染测试集的规则。测试计划需要同时覆盖“系统能否运行”和“系统给出的评测结论是否可信”两个层面。

本文档参考软件测试计划的通用结构组织内容，包含测试范围、测试对象、测试策略、测试环境、测试数据、测试类型、准入准出、执行安排、风险控制和交付物。具体测试项以《需求分析.md》《系统设计.md》定义的业务流程、模块边界和验收标准为基础，并结合项目迭代中形成的接口合同、测试资产和交付方式进行细化。

2. 测试目标与质量目标

2.1 总体目标

TRACE 测试工作的总目标是证明系统能够稳定完成测试生成、执行、追踪和评测闭环，并且在出现异常时给出可追溯、可解释的结果。测试结果应能支撑项目验收，也能为后续测试报告提供可量化数据。

2.2 功能目标

功能层面，测试应覆盖项目创建、快照管理、运行配置、策略版本、测试计划、测试运行、异步任务、报告导出、数据集管理、实验创建、实验启动、metrics 查询、artifact 查询、cleanup 和前端展示链路。每个主要接口都需要验证正常路径、参数错误、资源不存在、状态冲突和权限或路径边界。

智能体层面，测试应覆盖 Direct、Plan-and-Execute、ReAct + Reflection 三类策略，验证项目分析、上下文构建、提示词输入、结构化输出、Contract Guard、失败上下文回灌、一次反思修复和最终报告。重点关注模型输出不稳定、JSON 结构不合法、目标漂移、弱断言、未知 fixture 和 request/response 字段无证据等场景。

评测层面，测试应证明 clean run、clean replay、variant replay、sampled mutation score、capture matrix 和 EvaluationEvent 的计算口径正确。无效测试集、provider failure、artifact hash mismatch、setup failure、executor failure 和 flaky clean replay 必须与 0 捕获区分开。replay 只能执行冻结测试集，不能调用 LLM。

交付层面，测试应证明本地开发环境和 Docker Compose 环境均可启动、运行和验证。最终测试报告需要包含后端测试结果、前端构建结果、Compose smoke、benchmark suite、UI 彩排、已知风险和剩余缺口。

2.3 质量指标

以下为可量化的测试质量目标，测试准出时须满足：

指标	目标值	说明
P0 用例通过率	100%	所有 P0 用例必须通过，无阻塞缺陷
P1 用例通过率	≥ 95%	未通过的 P1 用例须有书面规避方案
后端快速回归通过率	100%	quick 分组全部通过
前端生产构建	通过	<code>npm run build</code> 无类型错误，产物完整
Docker Compose 服务健康	5/5 服务 healthy	五服务全部正常启动
Benchmark replay <code>llm_calls</code>	= 0	证明 replay 阶段不调用 LLM
三策略 <code>metric_status</code>	全部为 ok	无因系统错误导致的无效指标

2.4 测试角色与职责

角色	职责
测试负责人 (加鹿鸣)	制定和维护本测试计划；确认分组执行顺序；汇总测试报告；评审缺陷优先级
后端开发 (肖弘基，加鹿鸣)	提供可测试的接口合同和迁移脚本；修复 P0/P1 缺陷；辅助环境准入排查
前端开发 (杜浩冬)	确保 <code>npm run build</code> 通过；配合 UI 彩排；辅助 Playwright 用例环境配置
运维/交付 (加鹿鸣)	维护 Docker Compose 配置和交付脚本；负责镜像缓存和网络环境准入

3. 测试范围

本轮测试覆盖 TRACE V1 至 V2.4 的全部交付能力。测试范围按系统架构划分为后端 API、数据模型、智能体运行时、工具层、执行器与运行配置、评测系统、数据集与 mutation、前端可视化、交付脚本和非功能质量。

测试域	主要对象	覆盖重点	代表性验证方式
后端 API	projects、test-plans、test-runs、experiments、runtime-profiles、llm-options、evaluation	请求校验、状态码、资源状态、错误分类、返回结构	FastAPI TestClient、接口集成测试
数据模型与迁移	SQLAlchemy models、Alembic versions、repositories	表结构、外键、JSON 字段、迁移顺序、种子数据	schema migration 测试、仓储测试

测试域	主要对象	覆盖重点	代表性验证方式
智能体运行时	orchestrator、strategies、prompts、structured output、report	三策略流程、结构化输出、反思修复、报告内容	MockLLM 端到端测试、策略单元测试
工具层	analyze、write_test、run_pytest、search、lsp、fs_tools	路径边界、AST 分析、pytest 解析、工具输出结构	工具单元测试、临时项目执行测试
执行与运行配置	local_subprocess、runtime profile、executor command、rootdir	独立工作目录、命令拼装、超时、网络策略、preflight	执行器测试、runtime profile 合同测试
评测系统	eval dataset、experiment runner、replay、metrics、events	clean/replay 分层、捕获率、误报率、invalid 分类、LLM 调用隔离	experiment runner 测试、benchmark smoke
数据集与 mutation	dataset authoring、readiness、seeded bug、variant、mutation discovery	task/bug/variant 合同、probe confirmation、operator 分母	dataset API 测试、mutation discovery 测试
前端可视化	RunConsole、Experiment、Dataset、Runtime、Comparison、Report	数据加载、状态展示、阻塞原因、cleanup 确认、导出入口	类型检查、生产构建、UI 彩排
交付脚本	docker-compose、compose_smoke、compose_benchmark	服务启动、初始化、smoke、benchmark JSON 输出	PowerShell 脚本执行、Compose 环境验证
非功能质量	安全、可靠性、可观测性、可复现性、兼容性	脱敏、幂等、日志字段、artifact hash、配置隔离	专项测试、代码审查、故障注入

以下能力不纳入本期测试通过标准：远程 Runner 注册和心跳、long-poll 调度、分布式 replay、MinIO/S3 artifact storage、完整 OpenTelemetry、复杂权限审批流、多语言测试生成和模型训练平台。相关能力可在后续版本中建立独立测试计划。

4. 测试策略

4.1 分层测试金字塔

测试策略采用分层组合方式。底层以单元测试和合同测试保证数据结构、工具输出和业务规则稳定；中层以接口测试和集成测试覆盖 API、数据库、异步任务和执行器；上层以端到端测试、benchmark 和 UI 彩排验证真实业务链路。对于智能体和评测能力，额外引入变异测试思想，通过 seeded bug 和 auto mutation 验证生成测试是否能捕获已知缺陷。

端到端 / Benchmark / UI 彩排	← 少量，高价值
MockLLM E2E、真实 LLM smoke、Compose smoke/benchmark	
接口与集成测试	
FastAPI TestClient、DB 事务、Celery eager、experiment runner、replay scheduler	← 中等数量
单元与合同测试	
工具函数、Contract Guard、mutation discovery、metrics 计算、runtime profile 合同、source context	← 大量

4.2 测试分组策略

为兼顾执行速度和覆盖深度，将后端测试划分为以下五个执行层次，分别对应不同触发时机：

层次	执行集合	覆盖范围	触发时机	预估耗时
快速回归 (quick)	G01 工具层、G02 API 基础、G03 Runtime/LLM、G07 剩余功能	工具、Guard、source context、mutation、API 基础、prompt、executor	每次提交前	< 10 分钟
集成回归 (integration)	G04 Dataset/Mutation、G05 Replay/Cleanup	Dataset API、runtime profile、mutation 分母、replay、cleanup	功能合并前	< 10 分钟
慢速集成 (stage4-slow)	G06 Experiment Runner	experiment runner 完整主路径、CLI service path、benchmark 类用例	阶段验收前	30 分钟以上
前端门禁 (frontend)	G09 前端构建与 UI 自动化	TypeScript 类型检查、Vite 生产构建、Playwright 端到端	合并前及发布前	< 5 分钟 (构建) + UI 用例
交付验证 (delivery)	G10 Docker Compose	五服务启动、smoke、benchmark	发布前	视网络和构建速度

4.3 MockLLM 与真实 LLM 策略

默认回归使用 MockLLM。MockLLM 提供确定性响应，适合验证系统流程、状态机和数据持久化。真实 LLM 只用于 smoke，需要单独标记，并与 MockLLM 回归结果分开记录。不允许用真实 LLM 的非确定性结果替代 MockLLM 流程回归结论。

4.4 测试数据隔离策略

测试数据分为三类，不能混用：

- **临时测试数据**：后端单元和接口测试使用临时数据库和 fixture 项目，每个测试独立构造，测试结束后清理。临时目录统一使用仓库内 `.pytest_tmp_reports` 或短路径 `D:\t*`，避免系统临时目录权限和 Windows 长路径问题。
- **评测演示数据**：`eval/demo` 的 demo dataset 用于评测链路测试，clean baseline 目录不应包含历史运行产物（`.trace_artifacts`、`.trace_junit.xml`、`.trace_pytest_report.json`），否则会在物化时触发路径过长。
- **交付验证数据**：Docker Compose 初始化数据和外部项目 smoke 数据，只用于交付环境验证，不作为默认回归依据。

4.5 前端测试策略

前端测试以类型检查、生产构建和 Playwright UI 自动化为主。构建门禁要求 `npm run build` 通过，关键页面通过浏览器级 Playwright 自动化驱动真实后端 `/api` 代理完成数据验证，不依赖 demo fallback 数据。Playwright 用例应固定界面语言（`localStorage trace-locale=en`），保证选择器稳定。

5. 测试类型与方法

5.1 单元测试

单元测试用于验证单个函数、类或服务方法的输入输出。后端单元测试使用 pytest 执行，重点覆盖不依赖完整服务启动的业务逻辑，例如路径策略、错误码、环境变量模板脱敏、运行配置规范化、source context 切片、Contract Guard 规则、mutation discovery operator、metrics 聚合和 EvaluationEvent policy。

单元测试需要采用等价类划分、边界值分析和异常路径覆盖。以 runtime profile 为例，local_subprocess 的 network policy、resource limit、replay policy 数值边界都应有正常值、缺省值、非法值和边界值用例。以 Contract Guard 为例，需要覆盖弱断言、缺 fixture 证据、方法路径漂移、request field 无证据、response model 字段不匹配和源码行为矛盾等类别。

5.2 接口测试

接口测试覆盖 FastAPI 对外路由，验证请求参数、响应结构、状态码和错误信息。项目、快照、测试计划、测试运行、策略版本、运行配置、数据集、实验、LLM option、cleanup 和 replay cache 都应具备接口测试。

接口测试需要覆盖正常创建、查询列表、查询详情、更新、归档、资源不存在、状态冲突、字段非法、引用关系错误和重复请求等场景。对于创建 experiment、启动 experiment、清理 workspace、confirm mutation 等高风险操作，应验证服务端不会信任前端派生状态，必须重新校验 dataset readiness、runtime profile 归属、LLM option 可选性和 mutation discovery 结果。

5.3 集成测试

集成测试用于验证多个模块协同后的行为。TRACE 的核心集成场景包括 API + 数据库、API + Worker、Worker + Agent、Agent + tools、tools + pytest、experiment runner + replay scheduler、cleanup + artifact inventory、source context + Contract Guard。

集成测试应尽量使用真实数据库事务和临时文件目录，避免用过多 mock 掩盖模块边界问题。异步任务可以通过 Celery eager 模式执行，以保证测试速度和可重复性。涉及真实 LLM 的集成测试应单独标记为 smoke，不进入默认快速回归。

5.4 端到端测试

端到端测试验证用户视角的完整链路。普通测试生成链路应覆盖：创建项目，创建快照，创建测试计划，触发 run，等待 completed 或 failed，查看 trace、pytest results、artifacts 和 report。评测链路应覆盖：选择 dataset，创建三策略 experiment，启动 draft experiment，等待 completed，查看 metrics、EvaluationEvent、Runtime evidence、Replay evidence、Report/Export 和 cleanup dry-run。

端到端测试分为 MockLLM 和真实 LLM 两类。MockLLM 用于稳定验证系统流程、状态机和数据持久化；真实 LLM 用于验证模型配置、输出结构、Contract Guard retry、token 限制和真实失败形态。真实 LLM smoke 需与 MockLLM 结果分开记录，避免把确定性流程验证和模型能力验证混为一个结论。

5.5 回归测试

回归测试在每个阶段收尾、主链路合并、缺陷修复和交付前执行。后端回归以 `backend/tests` 全量 pytest 为主；前端回归以 `npm run build` 为主；交付回归包括 Compose smoke 和 benchmark suite。对数据库迁移、experiment runner、Contract Guard、runtime profile、mutation denominator、cleanup 和 LLM option 的改动，应追加目标测试后再执行全量回归。

回归测试需要保存命令、执行时间、通过数、失败数、跳过数、warning 类型和失败摘要。测试报告中应说明本次回归覆盖了哪些模块，哪些测试因外部环境缺失被跳过，哪些 warning 属于已知非阻塞问题。

5.6 评测有效性测试

评测有效性测试是 TRACE 的重点。测试需要验证 clean run、clean replay、variant replay 的分层关系，确认 false positive 只来自 clean baseline 失败，captured bug 只来自 bug variant replay 的有效 assertion failure。provider failure、setup failure、executor failure、artifact hash mismatch、invalid test set 和 flaky clean replay 均不能计为 0 捕获。

sampled mutation score 的测试应覆盖普通 seeded bug、auto mutation probe passed、auto mutation probe failed、缺失 probe_check、非 auto mutation、未完成 replay 等情况。分母只允许 confirmed 且 probe passed 的 auto mutation variant 进入。mutation operator 测试需要覆盖 comparison negation、arithmetic operator、boolean negation、constant replacement、return value 等已支持类型，并验证服务端 rediscovery 和 probe confirmation。

5.7 安全测试

安全测试重点覆盖路径边界、执行边界、配置脱敏和危险输入处理。Agent 工具只能在白名单目录内读写，`write_test_file` 不允许写业务源码、越权目录或绝对路径逃逸。`read_file` 和 `search_code` 需要限制文件大小和返回内容，避免把整个仓库或敏感文件塞给模型。

执行器安全测试需要验证 local_subprocess 的工作目录隔离、超时、命令参数拼接和 rootdir 策略。Docker 相关测试需要验证 Docker 不可用时能返回 preflight warning 或 executor unavailable，环境问题不得计入捕获率。LLM 配置错误响应必须脱敏，不回显密钥、base_url、本地路径或原始坏配置。

5.8 可靠性与异常测试

可靠性测试覆盖状态机、幂等、重试、取消和错误恢复。`test_runs.status` 与 `stage` 的组合必须符合生命周期设计，系统错误发生时先保存 `run_events` 和 `trace_steps`，再更新最终状态。`retry` 应创建新的 `run` 或新的受控 `attempt`，不能覆盖历史运行。

异常测试应覆盖非法模型输出、JSON 结构不合法、`pytest collection error`、`pytest execution error`、依赖安装失败、`Runner timeout`、`artifact` 写入失败、数据库冲突、重复 `result callback`、`cleanup` 越界路径和 `replay cache stale` 等场景。每个异常都应产生明确错误码、可读错误摘要和可追溯事件。

5.9 性能与容量测试

性能测试以本期交付规模为基准，不做大规模压测承诺。需要关注 API 查询响应、`experiment metrics` 聚合、`replay scheduler` 执行时间、`artifact inventory` 扫描时间、`cleanup dry-run` 时间和前端页面渲染体验。三策略 `benchmark repeat=3` 可作为基础容量样本，用于观察 `clean runs`、`replay runs`、`variant replay records` 和 `metrics` 聚合耗时。

后续若进入 V3 或大规模 `replay`，需要增加分页、`projection cache`、队列长度、Worker 并发、数据库索引和 `artifact storage` 容量测试。本期测试报告可记录当前数据量下的耗时和瓶颈，为后续容量规划提供依据。

5.10 兼容性与交付测试

兼容性测试覆盖 Windows 本地开发环境、Python 版本、Node.js 版本、PostgreSQL、Redis 和 Docker Desktop。Docker Compose 测试需要验证 `postgres`、`redis`、`api`、`worker`、`frontend` 五个服务可启动，前端可访问，API 可响应，`worker` 能执行实验任务。

交付测试包括基础 `smoke`、带 `experiment` 的 `worker/eval smoke`、三策略 `benchmark` 和真实 LLM `option` 读取。Docker 不可用、镜像缺失、`daemon` 未启动等环境问题应作为交付环境状态记录，不进入策略捕获率分母。

6. 测试环境

6.1 环境分类

测试环境分为本地开发环境、自动化回归环境、Docker Compose 交付环境和真实 LLM `smoke` 环境。不同环境的目标不同，本地环境侧重快速调试，自动化回归侧重稳定通过，Compose 环境侧重交付复现，真实 LLM 环境侧重模型接入和输出稳定性。

环境	用途	基础组件	说明
本地开发环境	单元测试、接口测试、目标回归	Python 3.10+、 <code>pytest</code> 、PostgreSQL、Redis、Node.js、 <code>npm</code>	适合开发阶段快速执行后端和前端检查
自动化回归环境	阶段验收、合并前回归	Python、 <code>pytest</code> 、PostgreSQL、Redis、Node.js	应固定依赖版本和数据库初始化脚本
Docker Compose 环境	软件交付和演示验证	PostgreSQL、Redis、Backend API、Worker、Frontend	使用 <code>docker compose up --build</code> 启动

环境	用途	基础组件	说明
真实 LLM smoke 环境	模型配置和真实输出验证	后端服务、Worker、真实 LLM 配置	密钥从 <code>.env</code> 或 <code>llm.config.json</code> 注入，结果单独记录

6.2 环境准入检查清单

进入测试执行阶段前，须完成以下环境准入检查：

```
# 基础组件版本检查
python --version          # 期望 3.10 以上
node --version            # 期望 v18 以上
npm --version
docker --version
docker compose version

# 服务可用性检查（本地环境）
# PostgreSQL 和 Redis 已启动且可连接
python -m pytest backend/tests/test_db_models.py --collect-only -q

# 前端依赖检查
cd frontend && npm install && npm run build --dry-run

# Docker Compose 交付环境检查
docker compose ps         # 应显示五服务状态
curl http://127.0.0.1:8000/healthz
curl http://127.0.0.1:5186/
```

已知准入问题与规避措施：

问题	原因	规避措施
pytest 临时目录 <code>PermissionError</code>	系统临时目录权限不足 (Windows)	统一使用 <code>--basetemp=D:\tn</code> 或仓库内 <code>.pytest_tmp_reports*</code>
Docker Hub 镜像拉取超时	外部网络不稳定	提前缓存 <code>node:20-alpine</code> 、 <code>nginx:1.27-alpine</code> 、 <code>python:3.12-slim</code>
<code>entrypoint.sh</code> CRLF 行尾	Windows 检出行尾转换	已通过 <code>.gitattributes</code> 强制 LF，无需额外处理
stage4 用例超过默认超时	完整实验运行耗时长	使用 <code>--timeout=600</code> 或分片执行

6.3 环境变量与密钥管理

环境变量和密钥文件不进入版本库。`.env`、`.env.local`、`backend/llm.config.json` 只在本地或交付环境保存。测试报告不得输出真实 API key、完整 base_url、敏感路径或未脱敏配置。LLM 配置错误时，服务端响应必须脱敏处理。

7. 测试数据计划

测试数据需要保证可复现、可隔离、可清理。后端单元和接口测试使用临时数据库和 fixture 数据，每个测试应尽量独立构造项目、快照、策略、运行配置和实验记录。文件系统测试使用临时目录，测试结束后清理生成文件。

评测数据以 demo benchmark suite 为主。数据集至少包含多个 eval task、seeded bug 和 bug variant，覆盖边界条件、比较符翻转、状态码错误、漏校验、返回值错误、布尔逻辑错误等缺陷类型。auto mutation 数据必须带 discovery evidence、patch evidence、probe metadata 和 probe_check 状态。

LLM 测试数据分为 MockLLM fixture 和真实模型响应。MockLLM fixture 要稳定、可重复，适合默认回归；真实模型响应用于 smoke 和人工分析，不作为默认回归的强依赖。真实 LLM 失败时，应记录 provider、模型名、错误分类、是否重试、是否进入 invalid_test_set 或 experiment failed。

前端演示数据只用于静态展示或 UI 开发，不作为真实 API 测试结果。Comparison 页面在 API 模式下只能消费 completed experiment 的真实 metrics，API 失败时应显示错误或空状态。

8. 测试用例设计

8.1 用例编号规则与模板

测试用例按模块建立编号和追踪关系，格式为 `TRACE-测试域-编号`。每条用例包含以下字段：

字段	说明
用例编号	唯一标识，如 <code>TRACE-API-001-TC01</code>
用例名称	简要描述测试意图
优先级	P0 / P1 / P2 / P3
前置条件	执行本用例所需的系统状态
测试数据	输入参数、fixture 文件或 mock 配置
测试步骤	操作序列（编号）
预期结果	每步或整体的期望输出
自动化状态	已自动化 / 计划自动化 / 手工
关联需求	对应测试计划编号或需求 ID

8.2 核心测试矩阵概览

编号	模块	测试重点	优先级	建议自动化
TRACE-API-001	项目与快照	创建、查询、归档、快照不可变、路径合法性	P0	是

编号	模块	测试重点	优先级	建议自动化
TRACE-API-002	测试计划	target_scope、budget、默认策略、非法字段校验	P0	是
TRACE-RUN-001	测试运行	queued/running/completed/failed/cancelled 状态迁移	P0	是
TRACE-RUN-002	retry/cancel	cancel 幂等、retry 创建新 run、历史结果不覆盖	P0	是
TRACE-AGENT-001	Direct 策略	单任务生成、结构化输出、报告生成	P0	是
TRACE-AGENT-002	Plan 策略	任务拆解、多 item 生成、target 规范化	P0	是
TRACE-AGENT-003	ReAct + Reflection	失败上下文回灌、一次修复、禁止削弱断言	P0	是
TRACE-TOOL-001	analyze_project	AST 分析、路由识别、函数/模型/测试识别	P0	是
TRACE-TOOL-002	write_test_file	受控目录写入、路径逃逸拒绝、hash 记录	P0	是
TRACE-TOOL-003	run_pytest	case result 解析、collection error、timeout、JUnit artifact	P0	是
TRACE-CTX-001	source context	target/dependency/support/failure/trace evidence 完整性	P0	是
TRACE-GUARD-001	Contract Guard	弱断言、目标漂移、fixture 缺失、字段无证据拒绝	P0	是
TRACE-LLM-001	LLM 配置	MockLLM、真实 LLM option、配置脱敏、provider failure	P0	是，真实模型为 smoke
TRACE-EVAL-001	dataset	task/bug/variant 创建编辑、readiness issue	P0	是
TRACE-EVAL-002	experiment	创建、draft start、状态冲突、runtime binding	P0	是
TRACE-EVAL-003	clean/replay	clean false positive、variant capture、llm_calls=0	P0	是
TRACE-EVAL-004	metrics	capture_rate、false_positive_rate、metric_status、cost	P0	是
TRACE-MUT-001	mutation discovery	comparison、arithmetic、boolean、constant、return operator	P1	是
TRACE-MUT-002	probe confirmation	confirmed/probe failed/probe missing 分母处理	P0	是

编号	模块	测试重点	优先级	建议自动化
TRACE-RUNTIME-001	runtime profile	executor、network、resource、replay_policy、preflight	P0	是
TRACE-CLEAN-001	cleanup	dry-run、二次确认、keep_failed、越界路径拒绝	P1	是
TRACE-FE-001	Run Console	stage rail、trace timeline、pytest table、report	P1	构建 + UI 彩排
TRACE-FE-002	Experiment Detail	start、metrics、events、runtime、cleanup、export	P0	构建 + UI 彩排
TRACE-FE-003	Dataset Detail	readiness、authoring、mutation confirmation、预选 experiment	P1	构建 + UI 彩排
TRACE-DEL-001	Compose smoke	五服务启动、API/Frontend 可访问、基础 smoke	P0	脚本
TRACE-DEL-002	Compose benchmark	三策略、repeat、metrics JSON、replay llm_calls=0	P0	脚本

8.3 分组测试设计详细说明

测试按执行分组组织，每组包含关联的测试文件、前置条件、执行命令和关键用例说明。分组编号 G01—G10 与第 9 节自动化执行框架对应。

G01 工具层 / Contract Guard / Source Context / Mutation / Eval Harness

关联测试计划编号：TRACE-TOOL-001/002/003、TRACE-CTX-001、TRACE-GUARD-001、TRACE-MUT-001、部分 TRACE-EVAL

测试目标：验证智能体执行闭环中最底层的确定性能力，包括代码分析是否准确、测试文件写入是否安全、pytest 输出是否可结构化解析、Guard 是否能拒绝不可信生成、source context 是否能提供足够证据、mutation 数据是否可被正确发现和确认。

前置条件：Python 环境可用；后端依赖已安装（`pip install -e backend`）；`eval/demo/clean` 目录不含历史运行产物。

计划执行命令：

```
python -m pytest \
  backend/tests/test_analyze.py \
  backend/tests/test_write_test_file.py \
  backend/tests/test_run_pytest.py \
  backend/tests/test_contract_guard.py \
  backend/tests/test_source_context.py \
  backend/tests/test_context_builder_guard_inputs.py \
  backend/tests/test_mutation_discovery.py \
  backend/tests/test_eval_harness.py \
  backend/tests/test_evaluation_events.py \
  -q --rootdir backend -p no:cacheprovider \
  --basetemp=D:\t3
```

关键用例：

用例编号	用例名称	前置条件	测试数据	预期结果
G01-TC01	项目 AST 和 FastAPI 路由分析	有效 Python 项目路径	test_analyze.py fixture 项目	返回含 routes/functions/models/existing_tests 的结构化对象
G01-TC02	生成测试写入越界路径拒绝	后端服务初始化	业务源码路径、父目录逃逸路径	越界写入返回错误，合法写入成功且记录 hash
G01-TC03	pytest 用例级结果解析	临时测试目录可写	含 passed/failed/skipped 的 pytest 项目	返回用例级结构化结果，collection error 与执行 error 分类明确
G01-TC04	Contract Guard 弱断言拒绝	source context 已构建	含 assert True、assert x is not None 的生成测试	弱断言被拒绝，violation 信息可回灌给反思提示词
G01-TC05	Contract Guard 目标漂移拒绝	目标函数路径已确定	测试调用与目标函数不同的函数路径	返回 target drift violation
G01-TC06	Contract Guard 未知 fixture 拒绝	无 conftest fixture 定义	生成测试使用了不存在的 fixture 名	返回 unknown fixture violation
G01-TC07	Source context 预算截断与 incomplete 标记	大型项目路径	超出 token 预算的依赖链	目标源码优先保留，截断部分标记为 incomplete，trace 可审计
G01-TC08	Mutation discovery comparison negation	含比较运算符的 Python 源码	> / < / == / != 等操作符	候选可发现，每个候选含操作符位置和变异后的代码
G01-TC09	Eval Harness clean/replay 分层	demo 数据集已初始化，clean baseline 无产物	eval/demo dataset	clean run 完成，variant replay 执行冻结测试集，llm_calls=0
G01-TC10	EvaluationEvent prompt-safe 与 audit-only 区分	实验已创建	触发 provider_failure 与 guard_reject 场景	prompt-safe 事件可暴露给前端，audit-only 事件仅内部记录

G02 API / 数据模型 / 基础集成

关联测试计划编号：TRACE-API-001/002、TRACE-RUN-001/002

测试目标：验证项目、快照、测试计划、测试运行、策略版本和数据库模型的正确性。重点验证快照路径必须位于 project root 之内，状态机迁移合法，retry 创建新 run 而不覆盖历史。

前置条件：PostgreSQL 和 Redis 可连接；后端数据库迁移已执行。

计划执行命令：

```
python -m pytest `
  backend/tests/test_api_local.py `
  backend/tests/test_api_strategy_versions.py `
  backend/tests/test_api_sync_run.py `
  backend/tests/test_api_async_run.py `
  backend/tests/test_api_retry_cancel.py `
  backend/tests/test_db_models.py `
  backend/tests/test_schema_migrations.py `
  backend/tests/test_sqlalchemy_recorder.py `
  backend/tests/test_report_sqlalchemy.py `
  backend/tests/test_v2_phase0_contracts.py `
  backend/tests/test_v2_versioning_api.py `
  -q --rootdir backend -p no:cacheprovider `
  --basetemp=D:\17524\MyCode\Trace\.pytest_tmp_reports\group2
```

关键用例：

用例编号	用例名称	前置条件	预期结果
G02-TC01	项目外路径创建 snapshot 应被拒绝	临时项目目录已创建	POST 快照请求携带 project root 外路径，返回 HTTP 400，错误描述包含 "snapshot root must be inside project path"
G02-TC02	项目内路径创建 snapshot 正常	临时项目目录已创建	POST 快照请求携带 project root 内路径，返回 201，快照可查询
G02-TC03	测试运行状态机迁移	测试项目和策略已创建	run 状态按 queued → running → completed/failed 迁移，不出现非法中间状态
G02-TC04	cancel 操作幂等	run 处于 running 状态	两次 cancel 请求均返回成功，run 最终状态为 cancelled，不抛出冲突错误
G02-TC05	retry 创建新 run	原 run 已 failed	retry 后产生新 run 记录，原 run 历史保留，新 run id 不同
G02-TC06	数据库迁移幂等性	已执行迁移	重复执行 Alembic upgrade 不报错，表结构与模型一致

G03 Runtime Profile / LLM 配置

关联测试计划编号：TRACE-RUNTIME-001、TRACE-LLM-001

测试目标：验证执行合同和模型配置边界，包括 project-scoped 与 dataset-scoped runtime profile、Docker executor 合同、local_subprocess 安全边界、replay policy 非法值处理、LLM option 可选性和配置脱敏。

计划执行命令：

```
python -m pytest `
  backend/tests/test_stage1_versioning_runtime_profiles.py `
  backend/tests/test_llm_options_api.py `
  backend/tests/test_llm_runtime_config.py `
  -q --rootdir backend -p no:cacheprovider `
  --basetemp=D:\17524\MyCode\Trace\pytest_tmp_reports\group3
```

关键用例：

用例编号	用例名称	预期结果
G03-TC01	Docker executor 缺少 image 字段应被拒绝	返回 400 或 validation error，错误信息指明 image 必填
G03-TC02	local_subprocess replay policy 并发数非法值	非法值（如负数、超界）返回 failed preflight 或 validation error
G03-TC03	LLM option API 不暴露不可选配置	只返回 selectable=true 的配置，不可选配置不出现在列表中
G03-TC04	LLM 配置错误响应不含密钥明文	触发 provider failure 时，错误响应中不包含 api_key、base_url 或原始坏配置

G04 Dataset / Mutation / 分母合同

关联测试计划编号：TRACE-EVAL-001、TRACE-MUT-001/002

测试目标：验证 Dataset authoring、seeded bug、variant、mutation discovery、probe confirmation 和 sampled mutation score 分母规则。

计划执行命令：

```
python -m pytest `
  backend/tests/test_eval_dataset.py `
  backend/tests/test_stage3_evaluation_seed_and_api.py `
  backend/tests/test_mutation_discovery.py `

  backend/tests/test_stage4_experiment_runner.py::test_sampled_mutation_score_denominator_requires_confirmed_probe_passed `
  -q --rootdir backend -p no:cacheprovider `
  --basetemp=D:\17524\MyCode\Trace\pytest_tmp_reports\group4
```

关键用例：

用例编号	用例名称	预期结果
G04-TC01	Dataset readiness 缺 task 返回 issue	readiness 响应包含结构化 issue，不返回空 issues
G04-TC02	普通 variant 不绕过 auto mutation confirmation	普通 variant 不要求 probe confirmation，但 auto mutation variant 必须经过 confirm 才能进分母
G04-TC03	probe failed 的 auto mutation variant 不进分母	sampled_mutation_score 分母不含 probe_check=failed 的 variant
G04-TC04	probe_check 缺失的 variant 不进分母	未执行 probe confirmation 的 auto mutation variant 不进分母
G04-TC05	多次 seed demo dataset 不漂移	重复执行 seed 操作，patch metadata 一致，不产生重复 variant

G05 Replay / Cleanup

关联测试计划编号：TRACE-CLEAN-001、部分 TRACE-EVAL-003

测试目标：验证 replay scheduler、replay cache、artifact inventory、workspace cleanup 和 cleanup 安全边界。

计划执行命令：

```
python -m pytest `
  backend/tests/test_artifact_cleanup.py `
  backend/tests/test_replay_scheduler.py `
-q --rootdir backend -p no:cacheprovider `
--basetemp=D:\17524\MyCode\Trace\.pytest_tmp_reports\group5
```

关键用例：

用例编号	用例名称	预期结果
G05-TC01	Cleanup dry-run 返回候选列表不删除	dry-run 响应包含候选 workspace 列表，实际文件未被删除
G05-TC02	keep_failed 保留失败 replay workspace	cleanup 时，failed 状态的 replay workspace 不出现在删除列表中
G05-TC03	越界路径标记 unsafe 或拒绝	.pytest_tmp_experiments 外路径在 cleanup 中被标记为 unsafe，不执行删除
G05-TC04	replay cache stale 判定	runtime 变更、artifact 变更或 ground truth 变更后，对应 cache 条目状态变为 stale
G05-TC05	stale cache 不被复用	标记为 stale 的 cache 不用于下一次 replay，触发重新执行

G06 Stage4 Experiment Runner

关联测试计划编号：TRACE-EVAL-002/003/004、TRACE-RUN-001、TRACE-LLM-001

测试目标：验证 experiment 完整评测链路，包括 clean run、variant replay、metrics 聚合、EvaluationEvent、provider failure 处理、artifact hash 校验、pipeline reject、cache 命中/失效和 CLI service path 复用。

执行注意事项：本组用例含有慢速集成测试，完整实验主路径单项耗时约 9 分钟，CLI service path 约 5 分钟。建议不作为快速回归运行，单独分配执行窗口或拆分到独立 CI job，使用 --timeout=600 参数。

计划执行命令（完整主路径，单项）：

```
python -m pytest \

    backend/tests/test_stage4_experiment_runner.py::test_experiment_runner_creates_clean_r
uns_replays_and_db_metrics \

    -q --rootdir backend -p no:cacheprovider --basetemp=D:\s42 --timeout=600
```

关键用例：

用例编号	用例名称	优先级	预期结果
G06-TC01	完整实验 runner 主路径	P0	三策略 clean runs 完成，variant replays 完成，DB metrics 可查询，报告可导出
G06-TC02	artifact hash mismatch 使实验 failed	P0	生成测试集 artifact 被篡改后，experiment 状态变为 failed，不继续 replay
G06-TC03	provider failure 不计为 0 捕获	P0	clean run provider failure 触发 invalid_test_set EvaluationEvent，不写入捕获指标
G06-TC04	completed experiment 不原地重跑	P0	对 completed experiment 再次 start 返回 409，不修改已有结果
G06-TC05	replay llm_calls = 0	P0	所有 variant replay run 的 llm_calls 字段值为 0
G06-TC06	mid-run cancel 可观察	P1	running 状态中取消，run 最终进入 cancelled，不留悬挂进程
G06-TC07	experiment workspace 越界拒绝	P0	尝试将 workspace 路径指向 .pytest_tmp_experiments 外返回 400
G06-TC08	clean setup failure 标记 invalid_test_set	P0	setup 阶段失败时，该 task 被标记为 invalid_test_set，不进入捕获分母
G06-TC09	Contract Guard pipeline reject 进入 invalid_test_set	P0	Guard reject 导致测试集无效，触发对应 EvaluationEvent，不计为 0 捕获
G06-TC10	CLI service path 复用 experiment service	P1	CLI 调用路径与 API 路径使用同一 experiment service 实例，不重复初始化

G07 后端剩余功能

关联测试计划编号：TRACE-AGENT-001/002/003、搜索与 LSP、executor、prompt、E2E MockLLM

计划执行命令：

```
python -m pytest `
  backend/tests/test_ast_grep_search.py `
  backend/tests/test_rg_search.py `
  backend/tests/test_lsp_definition.py `
  backend/tests/test_lsp_references.py `
  backend/tests/test_generation_prompt_context.py `
  backend/tests/test_generation_retry_prompt.py `
  backend/tests/test_reflection_prompt_context.py `
  backend/tests/test_plan_execute_targets.py `
  backend/tests/test_e2e_mockllm.py `
  backend/tests/test_celery_app.py `
  backend/tests/test_executor_rootdir.py `
  backend/tests/test_executor_runtime_command.py `
  backend/tests/test_llm_openai.py `
  backend/tests/test_llm_openai_chat_compat.py `
  backend/tests/test_m1_foundation.py `
-q --rootdir backend -p no:cacheprovider `
--basetemp=D:\17524\MyCode\Trace\.pytest_tmp_reports\group7
```

关键用例：

用例编号	用例名称	预期结果
G07-TC01	Guard violation 转成反思修复动作	generation retry prompt 包含 violation 详情，不直接放行不可信生成
G07-TC02	失败上下文进入反思提示词	reflection prompt context 包含 pytest failure 详情和相关源码证据
G07-TC03	Plan target ref 规范化	Plan-and-Execute 策略的 target refs 经规范化后指向合法函数路径
G07-TC04	MockLLM E2E 普通闭环	使用 MockLLM 完成单次 run，状态 completed，trace 和 report 可查询
G07-TC05	executor rootdir 与命令拼装	executor 使用指定 rootdir 执行 pytest，命令参数拼装正确，不出现路径注入

G08 后端全量回归

测试目标：在阶段验收前执行，验证各分组修复后无回归引入。

执行方式：由于 stage4 慢速用例耗时较长，全量单命令串行执行不适合作为本地门禁。建议按以下顺序分批执行，或在 CI 中按分组并行触发：

```
quick 分组 → integration 分组 → stage4-slow 分组（独立 job）
```

如需单命令全量执行（仅适合验收场景），参考命令：

```
python -m pytest backend/tests \
  -q --rootdir backend -p no:cacheprovider \
  --basetemp=D:\17524\MyCode\Trace\pytest_tmp_reports\full_regression \
  --timeout=600
```

注意：stage4 完整实验主路径和 CLI service path 耗时均在 5 分钟以上，全量串行运行时长预计超过 30 分钟。

G09 前端构建与 UI 自动化

关联测试计划编号：TRACE-FE-001/002/003

测试目标：验证 TypeScript 类型正确、Vite 生产构建产物完整，以及浏览器级端到端评测闭环可在真实后端下运行。

前置条件（Playwright 用例）：Docker Compose 栈已启动且 trace-api 和 trace-frontend 均 healthy；
\$env:PLAYWRIGHT_BASE_URL 指向前端地址。

计划执行命令：

```
# 类型检查与生产构建
cd frontend
npm run build

# Playwright 端到端（需 Compose 栈已启动）
$env:PLAYWRIGHT_BASE_URL = "http://127.0.0.1:5186"
npm run test:e2e
```

关键用例：

用例编号	用例名称	前置条件	预期结果
G09-TC01	TypeScript 类型检查	-	vue-tsc --noEmit 零错误
G09-TC02	Vite 生产构建	-	dist/index.html、CSS bundle、JS bundle 生成，无构建错误
G09-TC03	Dataset 列表与详情加载真实 API 数据	Compose 栈 healthy	Dataset 列表数据来源为 /api/datasets，详情页 readiness 面板渲染，可进入创建实验入口
G09-TC04	端到端评测实验创建与完成	Compose 栈 healthy，MockLLM 配置	从 Dataset 预选进入创建实验，填写 id，启动 run，等待 completed，metrics 行 metric_status=ok，replay 11m_calls=0

G10 Docker Compose 交付

关联测试计划编号：TRACE-DEL-001/002

前置条件：Docker daemon 运行；基础镜像（node:20-alpine、nginx:1.27-alpine、python:3.12-slim）可拉取或已缓存；entrypoint.sh 行尾为 LF（由 .gitattributes 保证）。

计划执行命令：

```
# 启动五服务
docker compose up --build -d

# 健康检查
docker compose ps
curl http://127.0.0.1:8000/healthz
curl http://127.0.0.1:5186/

# Compose smoke（可带 experiment run）
powershell -ExecutionPolicy Bypass -File scripts\compose_smoke.ps1 -RunExperiment

# Compose benchmark（三策略，repeat=3）
powershell -ExecutionPolicy Bypass -File scripts\compose_benchmark.ps1 `
  -RepeatCount 3 `
  -OutputPath docs\evidence\compose-benchmark-suite-r3.json
```

关键验证点：

验证点	期望值
五服务状态	全部 healthy
API healthz	{"ok": true}
前端 HTTP 状态	200
Smoke 实验 metric_status	ok
Benchmark 三策略 metric_status	全部 ok
Benchmark replay_llm_calls	[0]（replay 不调用 LLM）

9. 自动化测试框架规划

9.1 后端自动化框架

后端自动化测试以 pytest 为核心。测试目录保持在 backend/tests，公共 fixture 放在 backend/tests/conftest.py，临时项目放在 backend/tests/fixtures。测试按照模块命名，例如 test_run_pytest.py、test_contract_guard.py、test_stage4_experiment_runner.py，便于按风险域选择性执行。

pytest marker 规划：

marker	说明	默认回归中
@pytest.mark.slow	单项耗时超过 60 秒的集成测试	不包含，需 <code>-m slow</code> 显式触发
@pytest.mark.smoke_llm	需要真实 LLM 的 smoke 测试	不包含，需 LLM 配置且显式触发
@pytest.mark.stage4	stage4 experiment runner 完整链路	不包含，独立 CI job

临时目录约定：

所有测试使用 `--basetemp` 参数指定临时目录，避免系统默认路径权限问题和 Windows 长路径问题。本地开发推荐使用仓库内 `.pytest_tmp_reports/<group>` 或短路径 `D:\tN`。CI 环境应指定工作区内的短路径。

9.2 前端自动化框架

前端使用 `@playwright/test`，配置文件为 `frontend/playwright.config.ts` (chromium, 串行, 超时 240 秒)。相关文件：

<code>frontend/playwright.config.ts</code>	运行配置
<code>frontend/e2e/evaluation-flow.spec.ts</code>	评测主流程端到端用例

Playwright 用例通过 `PLAYWRIGHT_BASE_URL` 环境变量指定目标前端地址（默认 `http://127.0.0.1:5186`），通过前端 `/api` 代理直连真实后端，使用 MockLLM，不依赖静态 demo 数据。用例在挂载前将界面语言固定为英文（`localStorage trace-locale=en`），保证选择器稳定。

npm scripts：

<code>"test:e2e": "playwright test",</code>
<code>"test:e2e:report": "playwright show-report"</code>

9.3 交付自动化

交付自动化由 PowerShell 脚本承担：

脚本	功能
<code>scripts/compose_smoke.ps1</code>	验证 Compose 栈基础服务可用，带 <code>-RunExperiment</code> 时执行一次完整 experiment run
<code>scripts/compose_benchmark.ps1</code>	生成三策略、可配置 repeat 的实验，输出 JSON 证据到指定路径

benchmark JSON 是评测指标的原始证据，测试报告中引用的 capture rate、false positive rate、token 消耗等数据必须来源于该 JSON，不允许手动填写。

10. 测试执行计划

10.1 阶段执行矩阵

测试执行按阶段推进，每个阶段有明确的必跑测试和验收目标：

阶段	必跑测试分组	验收目标
V1	G01（工具层/Guard/eval harness）、G02（API 基础）、G07（MockLLM E2E）	验证单机闭环：工具可用、API 可用、MockLLM 端到端完成
V2.1	G06（experiment runner 主路径）、G01（eval harness）	验证评测有效性：clean/replay 分层、capture 指标口径
V2.2	G03（runtime profile）、G01（Guard/source context）、G02（API）	验证 A/B 合同：执行平台合同、项目理解能力
V2.3	G04（Dataset/mutation）、G05（replay/cleanup）	验证可维护性：mutation 分母、replay cache、cleanup 安全
V2.4	G01—G07 分组执行 + G09（前端）+ G10（Compose）	验证最终交付：全链路通过

10.2 日常开发执行规范

日常开发中，开发者应先执行受影响模块的目标测试，再根据改动风险决定是否执行更大范围回归：

- 修改工具函数、Guard 规则、source context：执行 G01
- 修改 API 路由、数据模型、状态机：执行 G02
- 修改 runtime profile 或 LLM config：执行 G03
- 修改 Dataset、mutation 或分母规则：执行 G04
- 修改 replay scheduler 或 cleanup：执行 G05
- 修改 experiment runner：执行 G06（单项慢速用例）
- 修改数据库迁移、评测指标或安全边界：执行受影响分组 + G02 全量

10.3 分层流水线设计

建议在 CI 中按以下流水线触发，而非单命令全量串行：

流水线	分组	触发时机	超时
quick	G01 + G02 + G03 + G07	每次 push	15 分钟
integration	G04 + G05	PR 合并前	15 分钟
stage4-slow	G06	阶段验收前	45 分钟
frontend	G09	PR 合并前 + 发布前	10 分钟
delivery	G10	发布前	视网络和镜像缓存

11. 准入、准出与暂停恢复标准

11.1 测试准入标准

进入测试执行的条件：

条件	说明
功能代码自测完成	开发者已在本地运行目标测试，无 P0 级语法或运行时错误
接口合同和 schema 已明确	相关 Pydantic schema、Alembic migration 已提交
数据库迁移可执行	<code>alembic upgrade head</code> 无报错
必要测试数据可构造	fixture 项目、demo dataset、MockLLM 配置可用
运行环境依赖可启动	PostgreSQL、Redis 可连接，前端依赖已安装
真实 LLM 测试的额外条件	配置文件存在、密钥可用、模型调用预算已获批准

11.2 测试准出标准

阶段验收通过的条件：

条件	要求
P0 用例通过率	100%，无任何 P0 用例失败或跳过（除明确标记的环境依赖跳过）
P1 用例通过率	≥ 95%，未通过的 P1 用例有书面规避方案且不影响主链路
后端 quick 分组	全部通过
前端生产构建	<code>npm run build</code> 通过，TypeScript 零错误
Compose smoke (V2.4 阶段)	五服务 healthy，smoke experiment 完成且 <code>metric_status=ok</code>
Benchmark (V2.4 阶段)	三策略 <code>metric_status</code> 全部为 ok， <code>replay_llm_calls=[0]</code>
阻塞缺陷	全部修复或关闭
非阻塞缺陷	全部写入测试报告遗留问题章节

11.3 暂停与恢复条件

以下情况应暂停对应范围的测试：

- 环境不可用（PostgreSQL/Redis 宕机，Docker daemon 未启动）
- 数据库迁移失败且无法回滚
- 关键依赖无法安装（pip/npm 安装失败）
- 真实 LLM 服务不可用（仅影响 smoke_llm 分组，不影响 MockLLM 回归）
- 主链路 P0 用例连续失败且根因未明

恢复条件：完成环境修复、数据重建或缺陷修复，并重新执行受影响范围的目标回归后方可恢复。

12. 缺陷管理

12.1 缺陷严重级别

级别	定义	示例
P0	主链路不可用、数据损坏、错误指标污染、密钥泄露、执行边界失效	项目外路径可创建 snapshot；评测结果混入历史产物；LLM 配置回显密钥
P1	核心功能异常但有临时规避	Docker 容器因行尾问题无法启动；eval harness 在特定环境触发路径过长
P2	局部功能、展示或边界问题	某个 API 字段缺失默认值；前端错误提示信息不准确
P3	文案、样式或低风险优化	按钮文案措辞；表格列宽调整

12.2 缺陷记录模板

每条缺陷记录应包含以下字段：

字段	内容
缺陷编号	BUG-XXX
严重级别	P0 / P1 / P2 / P3
模块	对应系统模块或测试组
环境	本地 / Compose / CI
版本	触发问题的代码版本或 commit
复现步骤	逐步操作序列
预期结果	正确行为描述
实际结果	错误行为或错误信息
关联测试用例	对应 <code>TRACE-XXX</code> 编号
影响指标	是否影响 capture rate、false positive、metric_status 或 <code>llm_calls=0</code> 口径
修复方案	修改文件、改动内容摘要
修复版本	修复对应的 commit 或版本
回归结论	复测命令和结果

12.3 缺陷关闭原则

缺陷关闭前必须完成回归验证，对应自动化用例必须通过。状态机、评测指标、Contract Guard、runtime profile、cleanup 和安全边界类缺陷，除复现用例外，还应补充或更新自动化测试以防止回归。

13. 测试报告输出规范

测试报告应面向项目验收和后续维护，包含测试范围、环境、执行命令、测试结果、缺陷统计、质量评价、风险说明和遗留问题。报告可以用 Markdown 作为人工阅读版本，同时输出 JSON 作为机器可读版本。

建议报告结构如下表。

章节	内容
测试概述	测试版本、测试时间、测试负责人、测试范围
测试环境	操作系统、Python、Node.js、PostgreSQL、Redis、Docker、LLM 配置口径
执行摘要	后端通过数、跳过数、warning、前端构建结果、Compose smoke、benchmark 结果
功能测试结果	按模块列出通过、失败、阻塞和未测项
评测结果	dataset、strategy、repeat、clean runs、replay runs、capture rate、false positive、llm_calls
非功能测试结果	安全、可靠性、可复现性、交付、性能观察
缺陷统计	按严重级别、模块、状态统计缺陷
风险与遗留问题	未覆盖能力、环境限制、后续建议
附件	pytest 输出、build 日志、benchmark JSON、UI 截图、运行报告链接

测试报告中的指标必须保留原始来源。例如 benchmark 指标应来自 experiment metrics 或脚本输出 JSON；前端结果应来自 `npm run build` 或 UI 彩排记录；后端结果应来自 pytest 输出。人工结论需要明确依据，不能用概括性描述替代可复查数据。

14. 测试交付物

测试交付物包括测试计划、测试用例库、自动化测试脚本、测试数据、测试执行日志、缺陷记录、测试报告和演示证据。自动化测试脚本随代码仓库维护，测试数据和 fixture 应尽量可重复生成，真实密钥和本地环境配置不得作为交付物提交。

本项目至少应保留以下测试交付物：`backend/tests` 后端测试集、`eval/harness` 评测脚本、`eval/demo` demo 数据集、`scripts/compose_smoke.ps1`、`scripts/compose_benchmark.ps1`、前端构建脚本、V2.4 演示 runbook、benchmark JSON 输出和 UI 彩排截图。最终测试报告应引用这些交付物，并说明其覆盖的测试范围。

15. 风险与应对

15.1 风险矩阵

风险 ID	风险描述	概率	影响	风险等级	应对措施
R-01	真实 LLM 服务不稳定，导致 smoke 用例不确定性高	高	中	高	默认回归使用 MockLLM；真实 LLM 只做 smoke，结果单独记录，不与确定性回归混用
R-02	Docker Hub 镜像拉取超时，阻塞 Compose 交付测试	中	高	高	提前缓存 <code>node:20-alpine</code> 、 <code>nginx:1.27-alpine</code> 、 <code>python:3.12-slim</code> ；配置稳定镜像代理
R-03	Windows 行尾转换导致 shell 脚本在容器内失效	中	高	高	通过 <code>.gitattributes</code> 强制 <code>*.sh</code> 使用 LF；新增 shell 脚本时同步更新规则
R-04	stage4 慢速用例单命令串行超时，全量回归不稳定	高	中	高	使用分层流水线；stage4 独立 CI job；本地回归按分组执行
R-05	Windows 长路径导致 eval harness 物化失败	中	中	中	<code>materialize_clean</code> 忽略历史运行产物；测试使用短路径 <code>--basetemp=D:\tN</code>
R-06	评测数据污染：demo metrics 混入真实 API 结果	低	高	中	Comparison 页面强制消费真实 API；demo 数据仅用于静态展示；测试中验证 API 模式不 fallback demo
R-07	Playwright 用例依赖 Compose 栈，本地环境不一致	中	低	低	固定 <code>PLAYWRIGHT_BASE_URL</code> 环境变量；UI 用例通过前端 <code>/api</code> 代理而非硬编码后端地址
R-08	mutation discovery 发现数超出预期，影响 probe 执行时间	低	低	低	分母规则只允许 confirmed + probe passed 进入；probe 执行在 demo 规模下可控

15.2 剩余风险说明

当前版本不纳入测试通过标准的能力（见第 3 节范围定义）构成剩余风险。主要包括：远程 Runner 调度、分布式 replay、MinIO/S3 artifact storage、完整 OpenTelemetry 链路和多语言测试生成。这些能力在本期交付中不存在，相关测试可在后续版本建立独立测试计划后补入。

16. 结论

TRACE 的测试计划需要同时服务软件质量和评测可信度。常规测试保证 API、数据库、前端、执行器和交付栈稳定；评测专项测试保证生成测试的有效性结论可信；非功能测试保证路径边界、脱敏、幂等、可复现和环境限制有明确结果。按照本文档建立测试框架后，团队可以直接执行分层测试、收集自动化结果，并生成覆盖功能、工程质量和评测指标的项目测试报告。